

EXERCÍCIO PRÁTICO RESOLVIDO

Layouts responsivos, navegação e fundamentos de programação front-end

Disciplina: Programação para Internet

Professor Msc Elionai Magalhães

Objetivo do material

Servir como exemplo completo para acompanhamento em laboratório. O aluno construirá, passo a passo, uma página responsiva para o estudo de caso “Agenda Fácil”, aplicando HTML semântico, navegação, CSS mobile first, Flexbox, Grid e media queries. Em seguida, acrescentará comportamento com JavaScript para exercitar variáveis, funções, condicionais, repetição, DOM e eventos.

Resultado esperado: ao final, o estudante terá três arquivos funcionais — index.html, style.css e script.js — e será capaz de explicar por que o layout muda entre celular e computador e como as interações JavaScript respondem às ações do usuário.

1. Enunciado do exercício

Situação-problema. Um pequeno salão deseja disponibilizar uma página chamada Agenda Fácil para apresentar serviços, informar horários e receber solicitações de agendamento. O protótipo deverá funcionar localmente no navegador, adaptar-se a telas pequenas e grandes e possuir interações simples em JavaScript.

Requisitos a serem atendidos

- Criar uma página com header, nav, main, seções e footer, mantendo estrutura semântica.
- Usar meta viewport e abordagem mobile first para adaptação a diferentes larguras.
- Criar navegação por links internos para Serviços, Horários e Agendamento.
- Usar Flexbox e/ou Grid para reorganizar conteúdo conforme o espaço disponível.
- Criar pelo menos uma media query para melhorar a apresentação em telas maiores.
- Adicionar formulário com nome, e-mail, serviço, data e observação.
- Implementar uma interação para mostrar/ocultar horários e outra para exibir mensagem conforme o serviço selecionado.
- Usar JavaScript com const/let, funções, if/else, repetição, DOM e addEventListener.
- Testar a página em visualização móvel e desktop usando DevTools.
- Manter foco visível, textos claros e navegação por teclado.

Ferramentas: Visual Studio Code ou outro editor, navegador Chrome/Edge/Firefox e DevTools. Não é necessário servidor, framework ou biblioteca externa.

2. O que será praticado

Conteúdo	Aplicação no exercício	Exemplo
Responsividade	Adaptação da interface	viewport, mobile first, unidades relativas
Navegação	Links internos e ordem lógica	nav, href="#servicos"
Layout	Organização de blocos	Flexbox, Grid, gap
Media query	Mudança em telas maiores	@media (min-width: 48rem)
Variáveis	Guardar referências e valores	const, let
Funções	Reutilizar instruções	function alternarHorarios()
Condicionais	Escolher caminhos	if / else if / else
Repetição	Percorrer elementos	forEach
DOM e eventos	Alterar a página após ações	querySelector, addEventListener

3. Preparação do projeto

1. Crie uma pasta chamada agenda_facil.
2. Dentro dela, crie os arquivos index.html, style.css e script.js.
3. Abra a pasta no editor de código.
4. Salve os arquivos a cada etapa e atualize o navegador para verificar o efeito.

Estrutura final da pasta

```
agenda_facil/  
├── index.html  
├── style.css  
└── script.js
```

4. Passo 1 — Estrutura inicial e viewport

Código inicial de index.html

```
<!DOCTYPE html>  
<html lang="pt-BR">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Agenda Fácil — Salão Escola</title>  
  <link rel="stylesheet" href="style.css">  
  <script src="script.js" defer></script>  
</head>  
<body>  
  <!-- conteúdo será inserido nos próximos passos -->  
</body>  
</html>
```

O atributo lang indica o idioma do documento. O meta charset evita problemas de codificação. A meta viewport orienta o navegador móvel a usar a largura real do dispositivo. O atributo defer faz o JavaScript ser executado depois que o HTML foi analisado, reduzindo erros ao selecionar elementos do DOM.

Teste rápido: abra a página no navegador e use DevTools → Toggle device toolbar. Compare larguras como 390 px, 768 px e 1280 px.

5. Passo 2 — Cabeçalho e navegação responsiva

Cabeçalho e menu

```
<header class="topo" id="inicio">
  <div class="container cabecalho">
    <div>
      <p class="marca">Agenda Fácil</p>
      <p class="subtitulo">Protótipo acadêmico de agendamento</p>
    </div>

    <nav aria-label="Navegação principal">
      <ul class="menu">
        <li><a href="#servicos">Serviços</a></li>
        <li><a href="#horarios">Horários</a></li>
        <li><a href="#agendamento">Agendamento</a></li>
      </ul>
    </nav>
  </div>
</header>
```

Os links usam identificadores internos. Quando o usuário ativa “Serviços”, o navegador procura o elemento com id="servicos". Em uma versão mobile first, o menu começa em coluna; depois, uma media query o transforma em linha em telas maiores.

6. Passo 3 — Conteúdo principal e cards

Seção de serviços

```
<section class="secao container" id="servicos">
  <div class="secao-titulo">
    <p class="destaque">Serviços</p>
    <h2>Opções disponíveis</h2>
    <p>Os cards se reorganizam conforme a largura disponível.</p>
  </div>

  <div class="grade-servicos">
    <article class="servico">
      <span class="numero">01</span>
      <h3>Corte</h3>
      <p>Atendimento para corte feminino ou masculino.</p>
    </article>
    <article class="servico">
      <span class="numero">02</span>
      <h3>Coloração</h3>
      <p>Aplicação de coloração com avaliação prévia.</p>
    </article>
    <article class="servico">
      <span class="numero">03</span>
      <h3>Manicure</h3>
    </article>
  </div>
```

```
<p>Cuidados básicos com unhas das mãos.</p>  
</article>  
</div>  
</section>
```

7. Passo 4 — Horários com conteúdo que pode ser exibido

HTML da interação mostrar/ocultar

```
<section class="secao secao-alternativa" id="horarios">
  <div class="container duas-colunas">
    <div>
      <p class="destaque">Horários</p>
      <h2>Quando atendemos</h2>
      <button id="botao-horarios" type="button"
        aria-expanded="false"
        aria-controls="detalhes-horarios">
        Mostrar horários detalhados
      </button>
    </div>

    <div id="detalhes-horarios" class="detalhes oculto">
      <h3>Atendimento</h3>
      <ul>
        <li>Segunda a sexta: 9h às 18h</li>
        <li>Sábado: 9h às 13h</li>
        <li>Domingo: fechado</li>
      </ul>
    </div>
  </div>
</section>
```

O botão é um elemento nativo, portanto já funciona por mouse, toque e teclado. `aria-expanded` informará se o conteúdo está aberto; `aria-controls` liga o botão à região controlada. A classe `oculto` será removida e adicionada pelo JavaScript.

8. Passo 5 — Formulário de agendamento

Trecho principal do formulário

```
<form id="form-agendamento" class="formulario" action="#" method="post">
  <div class="campo">
    <label for="nome">Nome *</label>
    <input id="nome" name="nome" type="text" required minlength="3">
  </div>

  <div class="campo">
    <label for="email">E-mail *</label>
    <input id="email" name="email" type="email" required>
  </div>

  <div class="campo">
    <label for="servico">Serviço *</label>
    <select id="servico" name="servico" required
      aria-describedby="mensagem-servico">
```

```
<option value="">Selecione...</option>
<option value="corte">Corte</option>
<option value="coloracao">Coloração</option>
<option value="manicure">Manicure</option>
</select>
<small id="mensagem-servico" aria-live="polite">
  Escolha um serviço para ver uma orientação.
</small>
</div>
</form>
```

9. Código HTML completo — parte 1

index.html — início até a abertura da seção Horários

```

<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Agenda Fácil — Salão Escola</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
  <header class="topo" id="inicio">
    <div class="container cabecalho">
      <div>
        <p class="marca">Agenda Fácil</p>
        <p class="subtitulo">Protótipo acadêmico de agendamento</p>
      </div>
      <nav aria-label="Navegação principal">
        <ul class="menu">
          <li><a href="#servicos">Serviços</a></li>
          <li><a href="#horarios">Horários</a></li>
          <li><a href="#agendamento">Agendamento</a></li>
        </ul>
      </nav>
    </div>
  </header>

  <main>
    <section class="hero">
      <div class="container hero-conteudo">
        <div>
          <p class="destaque">Atendimento simples e organizado</p>
          <h1>Escolha o serviço e solicite um horário.</h1>
          <p>Esta página demonstra layout mobile first, navegação responsiva e interações básicas em JavaScript.</p>
          <a class="botao-link" href="#agendamento">Solicitar agendamento</a>
        </div>
        <div class="painel" aria-label="Resumo do protótipo">
          <strong>Conteúdos praticados</strong>
          <ul>
            <li>Flexbox e Grid</li>
            <li>Media query</li>
            <li>Variáveis e funções</li>
            <li>Condicionais e eventos</li>
          </ul>
        </div>
      </div>
    </section>
  </main>

```

```

    </div>
  </div>
</section>

<section class="secao container" id="servicos">
  <div class="secao-titulo">
    <p class="destaque">Serviços</p>
    <h2>Opções disponíveis</h2>
    <p>Em telas pequenas, os cards aparecem em uma coluna. Em telas maiores, a grade se
reorganiza automaticamente.</p>
  </div>
  <div class="grade-servicos">
    <article class="servico">
      <span class="numero">01</span>
      <h3>Corte</h3>
      <p>Atendimento para corte feminino ou masculino.</p>
    </article>
    <article class="servico">
      <span class="numero">02</span>
      <h3>Coloração</h3>
      <p>Aplicação de coloração com avaliação prévia.</p>
    </article>
    <article class="servico">
      <span class="numero">03</span>
      <h3>Manicure</h3>
      <p>Cuidados básicos com unhas das mãos.</p>
    </article>
  </div>
  <p class="ajuda">Dica: clique ou pressione Enter em um card para destacar a opção.</p>
</section>

<section class="secao secao-alternativa" id="horarios">

```

9. Código HTML completo — parte 2

index.html — continuação

```

<div class="container duas-colunas">
  <div>
    <p class="destaque">Horários</p>
    <h2>Quando atendemos</h2>
    <p>O conteúdo adicional começa oculto e é exibido por uma função JavaScript acionada por
evento de clique.</p>
    <button id="botao-horarios" type="button" aria-expanded="false" aria-controls="detalhes-
horarios">
      Mostrar horários detalhados
    </button>
  </div>
  <div id="detalhes-horarios" class="detalhes oculto">
    <h3>Atendimento</h3>
    <ul>
      <li>Segunda a sexta: 9h às 18h</li>
      <li>Sábado: 9h às 13h</li>
      <li>Domingo: fechado</li>
    </ul>
  </div>
</div>
</section>

<section class="secao container" id="agendamento">
  <div class="secao-titulo">
    <p class="destaque">Agendamento</p>
    <h2>Solicite um horário</h2>
    <p>Ao escolher um serviço, o JavaScript lê o valor do campo, usa uma estrutura condicional e
atualiza a mensagem abaixo.</p>
  </div>

  <form id="form-agendamento" class="formulario" action="#" method="post">
    <div class="campo">
      <label for="nome">Nome *</label>
      <input id="nome" name="nome" type="text" required minlength="3" autocomplete="name">
    </div>
    <div class="campo">
      <label for="email">E-mail *</label>
      <input id="email" name="email" type="email" required autocomplete="email">
    </div>
    <div class="campo">
      <label for="servico">Serviço *</label>
      <select id="servico" name="servico" required aria-describedby="mensagem-servico">
        <option value="">Selecione...</option>
        <option value="corte">Corte</option>
      </select>
    </div>
  </form>

```

```

    <option value="coloracao">Coloração</option>
    <option value="manicure">Manicure</option>
  </select>
  <small id="mensagem-servico" class="mensagem" aria-live="polite">Escolha um serviço para
ver uma orientação.</small>
</div>
<div class="campo">
  <label for="data">Data preferida *</label>
  <input id="data" name="data" type="date" required>
</div>
<div class="campo campo-largo">
  <label for="observacao">Observação</label>
  <textarea id="observacao" name="observacao" rows="4" maxlength="200"></textarea>
</div>
<div class="campo-largo">
  <button type="submit">Enviar solicitação</button>
  <p id="mensagem-envio" class="mensagem-envio" aria-live="polite"></p>
</div>
</form>
</section>
</main>

<footer class="rodape">
  <div class="container">
    <p>Agenda Fácil — protótipo acadêmico para Programação para Internet.</p>
  </div>
</footer>
</body>
</html>

```

Observação: o HTML completo já contém as regiões que serão controladas pelo JavaScript. A aparência ainda será simples até a criação do style.css.

10. Passo 6 — Começar o CSS com mobile first

Primeiro bloco de style.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
  line-height: 1.6;
  color: #1f2937;
  background: #f6f7fb;
}

.container {
  width: min(92%, 1080px);
  margin-inline: auto;
}

.cabecalho {
  display: flex;
  flex-direction: column;
  gap: 1rem;
  padding-block: 1rem;
}

.menu {
  display: flex;
  flex-direction: column;
  gap: .4rem;
  list-style: none;
  padding: 0;
}
```

Mobile first significa que as regras iniciais atendem a tela pequena. Observe que cabeçalho e menu começam em coluna. Não existe “CSS para celular” separado: a própria base do projeto já funciona em pouco espaço.

11. Grid responsivo para os serviços

Cards em uma coluna

```
.grade-servicos {
  display: grid;
  grid-template-columns: 1fr;
  gap: 1rem;
}
```

```
.servico {  
  padding: 1.2rem;  
  background: #ffffff;  
  border: 2px solid transparent;  
  border-radius: .8rem;  
}  
  
.servico.selecionado {  
  border-color: #7a244d;  
  background: #fbf3f7;  
}
```

12. Passo 7 — Media query para telas maiores

Mudanças a partir de 48rem

```
@media (min-width: 48rem) {
  .cabecalho {
    flex-direction: row;
    align-items: center;
    justify-content: space-between;
  }

  .menu {
    flex-direction: row;
  }

  .grade-servicos {
    grid-template-columns: repeat(3, 1fr);
  }

  .duas-colunas {
    grid-template-columns: 1.15fr .85fr;
  }

  .formulario {
    grid-template-columns: repeat(2, 1fr);
  }

  .campo-largo {
    grid-column: 1 / -1;
  }
}
```

O breakpoint não representa um modelo específico de celular. Ele representa o momento em que há espaço suficiente para reorganizar menu, cards, horários e formulário. Essa escolha deve ser confirmada observando o conteúdo no DevTools.

Comportamento esperado

Largura aproximada	Navegação	Cards de serviços	Formulário
390 px	links em coluna	1 coluna	1 coluna
768 px	links em linha	3 colunas	2 colunas
1280 px	links em linha	3 colunas	2 colunas

Importante: se o layout começar a “apertar” antes ou depois de 48rem, o breakpoint pode ser ajustado. O comportamento do conteúdo deve orientar a decisão.

13. Código CSS completo — parte 1

style.css — início

```
* {  
  box-sizing: border-box;  
}  
  
:root {  
  --fundo: #f6f7fb;  
  --superficie: #ffffff;  
  --texto: #1f2937;  
  --muted: #5b6472;  
  --destaque: #7a244d;  
  --destaque-escuro: #5f1839;  
  --borda: #d7dce5;  
}  
  
html {  
  scroll-behavior: smooth;  
}  
  
body {  
  margin: 0;  
  font-family: Arial, Helvetica, sans-serif;  
  color: var(--texto);  
  background: var(--fundo);  
  line-height: 1.6;  
}  
  
img {  
  max-width: 100%;  
  height: auto;  
}  
  
.container {  
  width: min(92%, 1080px);  
  margin-inline: auto;  
}  
  
.topo {  
  background: var(--superficie);  
  border-bottom: 1px solid var(--borda);  
}  
  
.cabecalho {  
  display: flex;  
  flex-direction: column;
```

```
gap: 1rem;
padding-block: 1rem;
}

.marca {
margin: 0;
color: var(--destaque);
font-size: 1.4rem;
font-weight: 800;
}

.subtitulo {
margin: .1rem 0 0;
color: var(--muted);
font-size: .9rem;
}

.menu {
display: flex;
flex-direction: column;
gap: .4rem;
margin: 0;
padding: 0;
list-style: none;
}

.menu a {
display: block;
padding: .7rem .9rem;
color: var(--texto);
text-decoration: none;
border-radius: .5rem;
}

.menu a:hover,
.menu a:focus-visible {
background: #f0e6eb;
outline: 3px solid transparent;
}

.hero {
background: linear-gradient(135deg, #ffffff 0%, #f1e5eb 100%);
}

.hero-conteudo {
display: grid;
gap: 1.4rem;
```

```
padding-block: 3rem;
}

h1, h2, h3 {
  line-height: 1.2;
}

h1 {
  margin: .4rem 0 1rem;
  font-size: clamp(2rem, 7vw, 3.2rem);
}

h2 {
  margin: .4rem 0 .8rem;
  font-size: clamp(1.6rem, 5vw, 2.2rem);
}

.destaque {
  margin: 0;
  color: var(--destaque);
  font-weight: 800;
  text-transform: uppercase;
  letter-spacing: .04em;
}

.botao-link,
button {
  display: inline-block;
  min-height: 44px;
  padding: .75rem 1rem;
  border: 0;
  border-radius: .6rem;
  background: var(--destaque);
  color: #ffffff;
  font: inherit;
  font-weight: 700;
  cursor: pointer;
  text-decoration: none;
}

.botao-link:hover,
button:hover {
  background: var(--destaque-escuro);
}

.botao-link:focus-visible,
button:focus-visible,
```

```
input:focus-visible,  
select:focus-visible,  
textarea:focus-visible,  
.servico:focus-visible {  
  outline: 3px solid #1d4ed8;  
  outline-offset: 3px;  
}  
  
.painel {
```

13. Código CSS completo — parte 2

style.css — continuação

```
align-self: center;
padding: 1.2rem;
background: rgba(255, 255, 255, .88);
border: 1px solid var(--borda);
border-radius: .8rem;
}

.secao {
padding-block: 3rem;
}

.secao-alternativa {
background: #ffffff;
border-block: 1px solid var(--borda);
}

.secao-titulo {
max-width: 760px;
margin-bottom: 1.5rem;
}

.grade-servicos {
display: grid;
grid-template-columns: 1fr;
gap: 1rem;
}

.servico {
padding: 1.2rem;
background: var(--superficie);
border: 2px solid transparent;
border-radius: .8rem;
box-shadow: 0 8px 24px rgba(31, 41, 55, .07);
cursor: pointer;
}

.servico.selecionado {
border-color: var(--destaque);
background: #fbf3f7;
}

.numero {
color: var(--destaque);
font-weight: 800;
```

```
}

.ajuda,
.mensagem,
.mensagem-envio {
  color: var(--muted);
  font-size: .92rem;
}

.duas-colunas {
  display: grid;
  gap: 1.5rem;
}

.detalhes {
  padding: 1.2rem;
  border: 1px solid var(--borda);
  border-radius: .8rem;
  background: var(--fundo);
}

.oculto {
  display: none;
}

.formulario {
  display: grid;
  grid-template-columns: 1fr;
  gap: 1rem;
  padding: 1.2rem;
  background: var(--superficie);
  border: 1px solid var(--borda);
  border-radius: .8rem;
}

.campo label {
  display: block;
  margin-bottom: .35rem;
  font-weight: 700;
}

input,
select,
textarea {
  width: 100%;
  padding: .75rem .8rem;
  border: 1px solid #8c95a3;
```

```
border-radius: .5rem;
background: #ffffff;
color: var(--texto);
font: inherit;
}

.mensagem {
  display: block;
  margin-top: .35rem;
}

.mensagem-envio {
  min-height: 1.6rem;
  margin: .7rem 0 0;
  font-weight: 700;
}

.rodape {
  padding-block: 1.5rem;
  background: #20242c;
  color: #ffffff;
}

@media (min-width: 48rem) {
  .cabecalho {
    flex-direction: row;
    align-items: center;
    justify-content: space-between;
  }

  .menu {
    flex-direction: row;
  }

  .hero-conteudo {
    grid-template-columns: 1.5fr .8fr;
    align-items: center;
  }

  .grade-servicos {
    grid-template-columns: repeat(3, 1fr);
  }

  .duas-colunas {
    grid-template-columns: 1.15fr .85fr;
    align-items: start;
  }
}
```

```
.formulario {  
  grid-template-columns: repeat(2, 1fr);  
  padding: 1.5rem;  
}  
  
.campo-largo {  
  grid-column: 1 / -1;  
}  
}
```

14. Passo 8 — Selecionar elementos e criar variáveis

Início de script.js

```
const botaoHorarios = document.querySelector("#botao-horarios");
const detalhesHorarios = document.querySelector("#detalhes-horarios");
const campoServico = document.querySelector("#servico");
const mensagemServico = document.querySelector("#mensagem-servico");
const formulario = document.querySelector("#form-agendamento");
const mensagemEnvio = document.querySelector("#mensagem-envio");
const cardsServico = document.querySelectorAll(".servico");
```

Cada `const` guarda uma referência a um elemento do DOM. `querySelector` retorna o primeiro elemento que corresponde ao seletor; `querySelectorAll` retorna uma coleção de elementos. Como essas referências não serão reatribuídas, `const` é adequado.

15. Passo 9 — Função e evento para mostrar/ocultar horários

Primeira interação JavaScript

```
function alternarHorarios() {
  const ficouOculto = detalhesHorarios.classList.toggle("oculto");
  const estaAberto = !ficouOculto;

  botaoHorarios.setAttribute("aria-expanded", String(estaAberto));
  botaoHorarios.textContent = estaAberto
    ? "Ocultar horários detalhados"
    : "Mostrar horários detalhados";
}

botaoHorarios.addEventListener("click", alternarHorarios);
```

A função concentra a responsabilidade de abrir e fechar a região. O evento `click` informa quando o usuário ativa o botão. `classList.toggle` alterna a classe `oculto` e retorna um valor booleano; esse valor é usado para atualizar o estado acessível e o texto do botão.

16. Passo 10 — Condicional e mensagem dinâmica

Segunda interação JavaScript

```
function criarMensagemServico(servico) {
  if (servico === "corte") {
    return "Corte: reserve aproximadamente 45 minutos para o atendimento.";
  } else if (servico === "coloracao") {
    return "Coloração: o salão fará uma avaliação antes de confirmar o horário.";
  } else if (servico === "manicure") {
    return "Manicure: reserve aproximadamente 40 minutos para o atendimento.";
  }

  return "Escolha um serviço para ver uma orientação.";
}

function atualizarMensagemServico() {
  const servicoEscolhido = campoServico.value;
  mensagemServico.textContent = criarMensagemServico(servicoEscolhido);
}

campoServico.addEventListener("change", atualizarMensagemServico);
```

O evento `change` ocorre quando a opção do `select` é alterada. A variável `servicoEscolhido` recebe o valor atual do campo. A função `criarMensagemServico` utiliza `if`, `else if` e `else` implícito no retorno final para decidir qual texto deve ser exibido.

Observe: o JavaScript não cria a mensagem apenas visualmente. O elemento possui `aria-live="polite"`, permitindo que uma atualização importante possa ser anunciada por tecnologias assistivas.

17. Passo 11 — Repetição com os cards

forEach aplicado a vários elementos

```
function destacarCards() {
  cardsServico.forEach((card) => {
    card.setAttribute("tabindex", "0");

    card.addEventListener("click", () => {
      card.classList.toggle("selecionado");
    });

    card.addEventListener("keydown", (evento) => {
      if (evento.key === "Enter" || evento.key === " ") {
        evento.preventDefault();
        card.classList.toggle("selecionado");
      }
    });
  });
}
```

```
}
```

```
destacarCards();
```

18. Passo 12 — Evento de envio do formulário

Interceptar submit no protótipo local

```
formulario.addEventListener("submit", (evento) => {
  evento.preventDefault();
  const nome = document.querySelector("#nome").value.trim();
  mensagemEnvio.textContent =
    `Solicitação registrada para demonstração, ${nome}.`;
});
```

preventDefault impede o envio real porque o exercício não possui servidor. A validação nativa do HTML continua sendo executada antes do evento submit. Em um sistema real, o back-end deverá receber e validar os dados; neste exemplo, o objetivo é demonstrar o comportamento no front-end.

19. Código JavaScript completo

script.js

```
const botaoHorarios = document.querySelector("#botao-horarios");
const detalhesHorarios = document.querySelector("#detalhes-horarios");
const campoServico = document.querySelector("#servico");
const mensagemServico = document.querySelector("#mensagem-servico");
const formulario = document.querySelector("#form-agendamento");
const mensagemEnvio = document.querySelector("#mensagem-envio");
const cardsServico = document.querySelectorAll(".servico");

function alternarHorarios() {
  const ficouOculto = detalhesHorarios.classList.toggle("oculto");
  const estaAberto = !ficouOculto;

  botaoHorarios.setAttribute("aria-expanded", String(estaAberto));
  botaoHorarios.textContent = estaAberto
    ? "Ocultar horários detalhados"
    : "Mostrar horários detalhados";
}

function criarMensagemServico(servico) {
  if (servico === "corte") {
    return "Corte: reserve aproximadamente 45 minutos para o atendimento.";
  } else if (servico === "coloracao") {
    return "Coloração: o salão fará uma avaliação antes de confirmar o horário.";
  } else if (servico === "manicure") {
    return "Manicure: reserve aproximadamente 40 minutos para o atendimento.";
  }

  return "Escolha um serviço para ver uma orientação.";
}

function atualizarMensagemServico() {
```

```
const servicoEscolhido = campoServico.value;
mensagemServico.textContent = criarMensagemServico(servicoEscolhido);
}

function destacarCards() {
  cardsServico.forEach((card) => {
    card.setAttribute("tabindex", "0");

    card.addEventListener("click", () => {
      card.classList.toggle("selecionado");
    });

    card.addEventListener("keydown", (evento) => {
      if (evento.key === "Enter" || evento.key === " ") {
        evento.preventDefault();
        card.classList.toggle("selecionado");
      }
    });
  });
}

botaoHorarios.addEventListener("click", alternarHorarios);
campoServico.addEventListener("change", atualizarMensagemServico);

formulario.addEventListener("submit", (evento) => {
  evento.preventDefault();
  const nome = document.querySelector("#nome").value.trim();
  mensagemEnvio.textContent = `Solicitação registrada para demonstração, ${nome}.`;
});

destacarCards();
```

20. Resultado esperado no navegador

A aparência pode variar ligeiramente conforme o navegador, mas a diferença entre as larguras deve ser evidente: no celular, menu, cards, seções e formulário ocupam uma coluna; no desktop, o conteúdo aproveita o espaço com menu horizontal, grade de serviços e campos distribuídos em duas colunas.

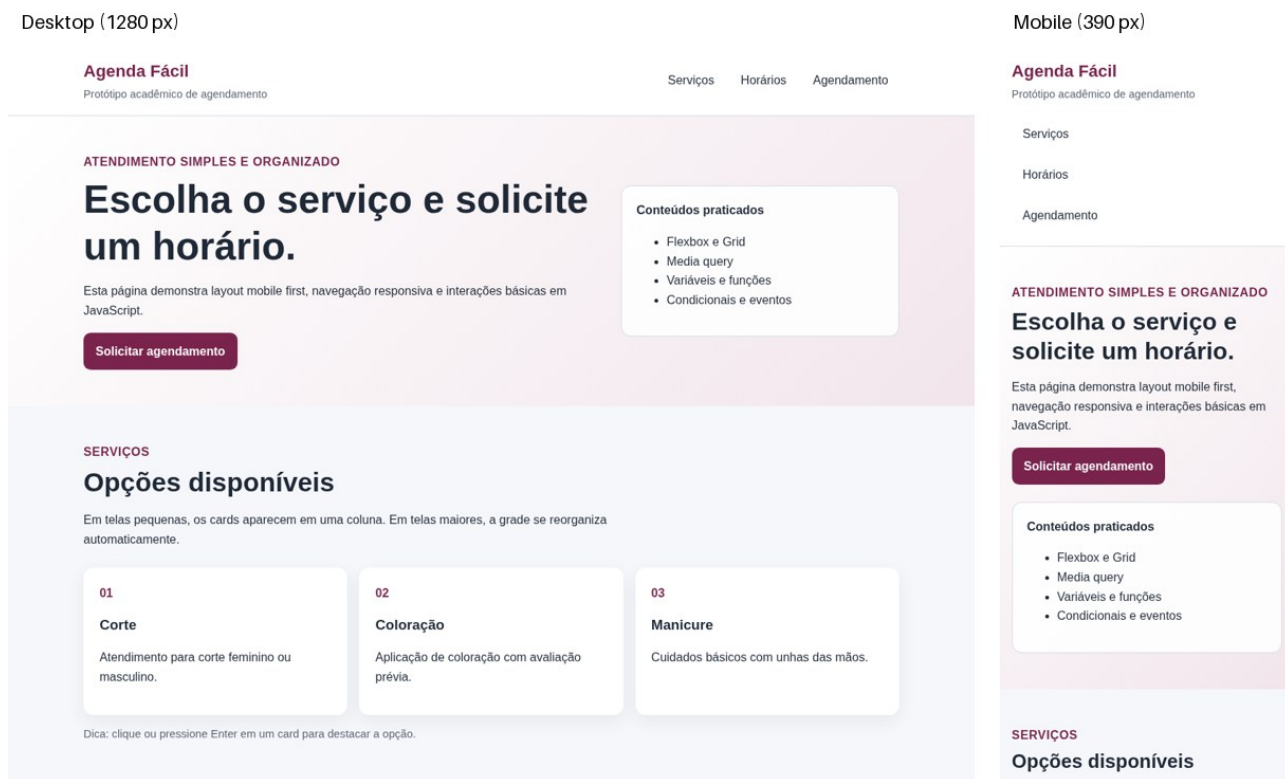


Figura 1 — Comparação do protótipo em desktop e mobile.

O que observar: a hierarquia visual permanece, não aparece rolagem horizontal, os links continuam acessíveis e o conteúdo essencial não desaparece na versão móvel.

21. Passo 13 — Testes que os alunos devem executar

Teste	Ação	Resultado esperado	Conceito
1	Abrir em 390 px	menu e cards em coluna; sem rolagem horizontal	mobile first
2	Abrir em 1280 px	menu em linha; 3 cards lado a lado	media query / Grid
3	Clicar em “Mostrar horários”	região aparece e texto do botão muda	função + click
4	Escolher “Cororação”	mensagem específica aparece	variável + if/else + change
5	Clicar ou pressionar Enter em card	card recebe/destroca destaque	forEach + eventos
6	Enviar formulário vazio	navegador impede o envio	validação HTML
7	Preencher formulário e enviar	mensagem de demonstração aparece	submit + preventDefault
8	Navegar apenas com Tab/Shift+Tab	foco visível e ordem lógica	acessibilidade
9	Redimensionar lentamente a janela	mudança ocorre sem quebrar conteúdo	breakpoint

22. Experimento adicional — DevTools

1. Abra a página no Chrome, Edge ou Firefox.
2. Pressione F12 ou use a opção Inspecionar.
3. Ative a visualização de dispositivo móvel.
4. Teste 390 px, 768 px e 1280 px e observe o ponto em que a media query entra em ação.
5. Na aba Elements/Inspector, desmarque temporariamente a regra grid-template-columns e veja como a grade muda.
6. Na aba Console, execute `document.querySelectorAll(".servico").length` e confirme quantos cards existem.

Pergunta para discussão: por que testar apenas um modelo de celular não é suficiente para afirmar que uma página é responsiva?

23. Resolução comentada — o que cada parte demonstrou

HTML semântico e navegação. Header, nav, main, section, article e footer organizam a página e os links internos permitem deslocamento entre áreas do protótipo.

Viewport e mobile first. A meta viewport usa a largura real do dispositivo; o CSS inicial funciona em telas pequenas e recebe melhorias progressivas.

Flexbox e Grid. Flexbox organiza cabeçalho e menu; Grid controla cards, colunas e formulário conforme o espaço disponível.

Media query. A regra min-width altera o layout quando há espaço suficiente, sem duplicar HTML.

Variáveis. Constantes guardam referências aos elementos do DOM e valores temporários usados nas interações.

Funções. Cada função possui responsabilidade definida: alternar horários, criar mensagem, atualizar texto ou aplicar eventos aos cards.

Condicionais. If/else seleciona mensagens diferentes conforme o serviço escolhido e também verifica teclas na interação com cards.

Repetição. forEach percorre todos os cards e aplica o mesmo conjunto de eventos sem duplicar código.

DOM e eventos. querySelector localiza elementos;.textContent, classList e setAttribute alteram estado; addEventListener responde a ações.

Acessibilidade. Foco visível, elementos nativos, navegação por teclado, aria-expanded, aria-controls e aria-live ajudam a manter a interface operável.

24. Checklist de conferência

- ☐ A página contém meta viewport.
- ☐ O menu possui links internos que funcionam.
- ☐ O CSS começa em uma coluna e melhora telas maiores.
- ☐ Existe pelo menos uma media query.
- ☐ Os cards usam Grid e não causam rolagem horizontal.
- ☐ O formulário possui labels e validação básica.
- ☐ O JavaScript está em arquivo externo e carregado com defer.
- ☐ Existem variáveis, funções e pelo menos uma estrutura condicional.
- ☐ O código usa repetição para percorrer elementos.
- ☐ As interações usam addEventListener.
- ☐ O botão de horários atualiza aria-expanded.
- ☐ A página funciona por teclado em largura pequena e grande.
- ☐ Os testes em DevTools foram executados e compreendidos.

25. Desafio opcional para continuar a prática

Depois de reproduzir a solução, faça uma cópia dos três arquivos e implemente as mudanças abaixo sem consultar o código completo. O objetivo é verificar se os conceitos foram compreendidos, e não apenas copiados.

- Transforme a navegação móvel em um menu que abre e fecha por botão, mantendo aria-expanded e uso por teclado.
- Adicione um quarto serviço e confirme que o Grid e o forEach continuam funcionando sem alterações estruturais no JavaScript.
- Crie uma função que conte quantos campos obrigatórios do formulário estão preenchidos e exiba um resumo.
- Crie um novo breakpoint somente se o conteúdo realmente precisar de reorganização. Justifique a escolha.
- Registre duas capturas: uma com largura móvel e outra em desktop, explicando as diferenças observadas.

26. Relação com a avaliação processual

O exercício serve como exemplo reduzido do tipo de protótipo solicitado na avaliação processual da disciplina. Ele integra HTML semântico, formulário, CSS responsivo e JavaScript básico com interações, mas não substitui o trabalho do estudante. No trabalho avaliativo, o grupo deverá adaptar conteúdo, identidade visual, estudo de caso e decisões técnicas, além de explicar as escolhas realizadas e apresentar evidências de funcionamento.

Atenção: o exemplo foi construído com o caso Agenda Fácil apenas para demonstração. Biblioteca Bairro Aberto, PetCare Local e Oficina Rápida podem aplicar o mesmo raciocínio com conteúdo e interações próprios.

27. Materiais de base utilizados

- Apresentação da disciplina: “Layouts responsivos e front-end” — responsividade, navegação, Flexbox, Grid, media queries, JavaScript, variáveis, funções, estruturas de controle, DOM e eventos.
- Exercício prático resolvido anterior: “Formulários HTML, validação, acessibilidade e CSS” — padrão de organização, passo a passo, código completo, testes e checklist.
- Orientações das Avaliações Processuais — 1º bimestre: protótipo web semântico, responsivo e interativo, com CSS responsivo e pelo menos duas interações em JavaScript.
- Material de aula: “Formulários, validação, acessibilidade e CSS” — base para formulário, foco, validação e boas práticas de acessibilidade.
- MDN Web Docs — Responsive Design, Media Queries, Flexbox, Grid, JavaScript, DOM e Events.
- W3C — Web Content Accessibility Guidelines (WCAG) 2.2 e Quick Reference.
- web.dev — Responsive Web Design Basics.

Observação ao professor

Uma estratégia de demonstração é projetar o editor ao lado do navegador. Implemente um bloco por vez, teste no DevTools e peça aos alunos que prevejam a mudança antes de atualizar a página. Na parte de JavaScript, abra o Console e mostre que erros de seleção de elemento ou sintaxe podem ser localizados durante a execução.